Lab Topic 5

Learning objectives

- Use a PSO code provided to you
- Apply PSO to implement GLRT for the quadratic chirp in colored Gaussian noise problem (review Lab 4)
- Supplementary:
  - Learn to use structures in Matlab
  - Learn to use function handles in Matlab

PSO codes

- Clone the repository: **GitHub →SDMBIGDAT19** (Store it outside GWSC)
- Lectures delivered at the BigDat19 5th International Winter school on Big Data, Cambridge University, UK (Jan, 2019)
- The codes and slides supplement the textbook; Cover PSO and stochastic optimization in greater depth (including fundamental theorems)

We will need the following codes in **SDMBIGDAT19/CODES**:

- **r2ss.m:** Helper function; no need to look inside
- **r2sv.m:** Helper function; no need to look inside
- **s2rs.m:** Helper function; no need to look inside
- **s2rv.m:** Helper function; no need to look inside
- **crcbchkstdsrchrng.m:** Helper function; no need to look inside
- crcbpso.m: Main PSO code that can be applied to any fitness function
- **crcbpsotestfunc.m:** A benchmark fitness function; Also an example for how to code fitness functions to work with crcbpso.m
- **crcbqcfitfunc.m:** The fitness function for quadratic chirp GLRT (in WGN)
- **crcbqcpso.m**: Applies PSO to the quadratic chirp fitness function
- **test_crcbpso.m:** Test function for crcbpso.m
- **test_crcbqcpso.m:** Test function for crcbqcpso.m

Exercise #1 Part 1

- Read the short user manual **CODES/CodeDoc.pdf**
- The main usage instructions are in the “help” for each function
- The **test_<funcName>.m** scripts show examples of usage for some of the functions
  - The **test_crcbpso.m** script shows how **crcbpso.m** is applied to a benchmark fitness function (defined in **crcbpsotestfunc.m**)

Exercise #1 Part 2

- To use the PSO code, the following are required
- Understand the concept of structures in Matlab
  - Matlab structures work in the same way as structures in C
  - X = struct('a', 5.0, 'b', 6.0);
  - disp(X.a) will show 5.0
  - disp(X.b) will show 6.0
- Structures offer a convenient way to move a large number of arguments into and out of a function
- Structures also help make your codes future-proof: New versions of codes can use new input arguments while old versions will ignore them

Tasks: Set 1

- Objective: Modify the function you wrote for generating your assigned signal to use a 'struct' as the input argument for specifying the signal parameters
- For example, if your function was called `foo(dataX, SNR, list_of_parameters)`, where '`list_of_parameters`' was either a vector (as in SIGNALS/`crcbgenqcsig.m`) or a set of input arguments, then pass the parameters inside a struct (`P`):
  - Modify `foo(dataX, SNR, list_of_parameters)` to `foo_new(dataX, SNR, P)`, where `P` is a struct
  - For example, if your parameters were a_1, a_2, a_3, the struct would be:
    - `P=struct('meaningful_name_of param1', ` a_1`, 'meaningful_name_of param2', ` a_2`,…)` where you should have field names that tell the reader of the code something useful about the nature of the parameter
  - You can look at **SDMBIGDAT19/CODES/`s2rv.m`** to see how a struct is passed to a function and how its is used inside it
- Compare the outputs of `foo_new` to `foo` for the same input values and make sure they are identical
- Create a test script to run `foo_new` that makes plots of (a) the signal time series with time axis in seconds, and (b) the signal periodogram for positive frequencies with the frequency axis in Hz

Exercise #1 Part 3

- Understand the concept of **function handle**
  - Encountered earlier: quadratic noise PSD
- Type "function handle" in Matlab's "Search documentation" bar and read more about this feature
- A function handle is a variable that can be used to call a function
- $z=5.0$
- $F = \underbrace{@(x,y)}_{\substack{\text{What are the input} \\ \text{variables that will be} \\ \text{sent to } F?}} \quad foo(x,z,y)$
- F is a handle to function foo
- $F(2.0, 3.0)$ is the same as $foo(2.0,\ 5.0,\ 3.0)$
- The CRCBPSO code accepts as input the handle to the fitness function to be optimized
- This allows the same PSO code to be run on any fitness function

Tasks: Set 2

- Write a script that does the following.
- Assigns values to `dataX` and the parameter struct, `P`, sent to `foo_new(dataX, SNR, P)`
  - For example:
    - `>> dataX = (0 : 499)/1024;`
    - `>> P = struct('meaningful_name_of param1', 5.0, 'meaningful_name_of param2', 3.0,…)`
  - Of course, the specific values of `dataX` and those of the fields of `P` will be different for your case
- Creates a function handle to the function `foo_new` :
  - `H = @(x) foo_new(dataX, x, P);`
- Uses the function handle to plot the time series of the signal for SNR values 10, 12, and 15

Anatomy of SDMBIGDAT19 codes

CRCBPSO: The PSO code

Inputs

- **Handle to fitness function**
  - Fitness function initialized with parameters that will not change from one call to another in PSO
  - Example: Data vector, PSD, sampling frequency
- **Number of search space dimensions** (= Number of parameters to maximize the likelihood Ratio over)
  - Example: 3 D search space for GLRT of quadratic chirp

Output

- **Structure** containing:
  - Best fitness value found
  - Location of the best fitness
  - Number of fitness evaluations (remember that particle fitness is not evaluated when they are outside the search space)

Notes

- Standardized coordinates:
  - CRCBPSO searches within the unit D-dimensional hypercube:
    $$x_i \in [0,1], \qquad i = 1,2, \dots, D$$
- Location of the best fitness returned in standardized coordinates
- Conversion to and from std. coord. is handled by the fitness function
- Each call to CRCBPSO is one run of PSO: Best-of-M-runs needs separate calls

CRCBPSO: The PSO code

% S=CRCBPSO(Fhandle,N)

% Runs local best PSO on the fitness function with handle Fhandle. If Fname

% is the name of the function, Fhandle = @(x) <Fname>(x, FP), where FP is

% the set of parameters needed by Fname.

Example: FP would be a structure that includes the data vector, PSD etc.

% N is the dimensionality of the

% fitness function.

%The output is returned in the structure S. The field

% of S are:

% 'bestLocation : Best location found (in standardized coordinates)

% 'bestFitness': Best fitness value found

% 'totalFuncEvals': Total number of fitness function evaluations.

Simple fitness function

Fitness functions should be able to handle multiple input locations

```
F = CRCBPSOTESTFUNC(X,P)
Compute the Rastrigin fitness function for each row of
X. X is standardized, that is 0<=X(i,j)<=1.
```

Example of the matrix X for a 2D space		
	Parameter 1	Parameter 2
Location 1	0.1	0.8
Location 2	0.5	0.2

```
The fitness values are returned in F.
```

Example of F for the above X	
	Fitness
Location 1	10.0
Location 2	2.0

Simple fitness function

```
F = CRCBPSOTESTFUNC(X,P)

P has two arrays P.rmin and P.rmax that are used to convert X(i,j) internally to
actual coordinate values before computing fitness:
```

$X(:,j) \; {-}{>} \; X(:,j) * (rmax(j) - rmin(j)) + rmin(j)$

Example for 2D space: P.rmin = [-5,5]; P.rmax = [5, 10] ⇒ range for parameter 1 is [-5,5] and for parameter 2 is [5,10]

- The fields rmin and rmax are required for any fitness function
- In your fitness function, the structure P will need more fields. Example: data, PSD, etc.

Part where the input standardized coordinates are converted to real coordinates

```
%Check for out of bound coordinates and flag them
validPts = crcbchkstdsrchrng(xVec);
%Set fitness for invalid points to infty
fitVal(~validPts)=inf;
%Convert valid points to actual locations
xVec(validPts,:) = s2rv(xVec(validPts,:),params);
```

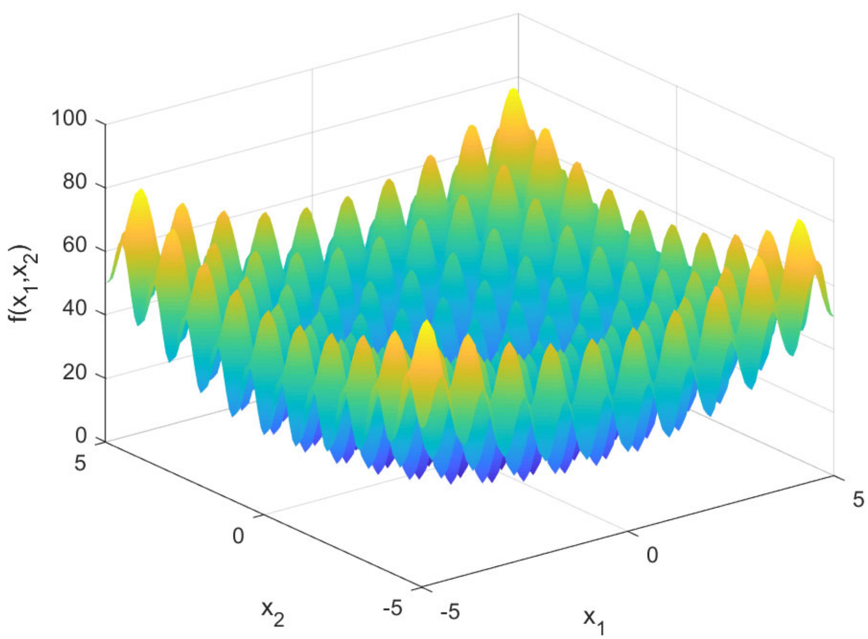
Simple fitness function

```
F = CRCBPSOTESTFUNC(X,P)
```

The main part of this function

```
for lpc = 1:nrows
    if validPts(lpc)
    % Only the body of this block should be replaced for different fitness
    % functions
        x = xVec(lpc,:);
        fitVal(lpc) = sum(x.^2-10*cos(2*pi*x)+10);
    end
end
```

Exercise #2



- Run **test_crcbpso.m**: it runs **crcbpso** on the standard benchmark fitness function **crcbpsotestfunc.m**
- Experiment:
  - Change number of dimensions
  - Change search range
- Optional challenge: Learn how to use the **surf** function in Matlab and make the plot shown (see Matlab documentation for surf and worked out examples)

Task: set 1

- Code a matlab function for any one benchmark fitness function following the example of **crcbpsotestfunc.m**
  - Ignore the fitness functions that have red highlights (there are typographical errors in them)
  - Column "D": Dimensionality of search space
  - Column "Feasible Bounds": Range of each coordinate defining the search space, which is a hypercube
- Apply **crcbpso** to the fitness function and find the solution for the global minimum and minimizer
- Experiment with different number of iterations (see additional features) and different number of runs in a best-of-M-runs strategy

TABLE I

BENCHMARK FUNCTIONS

Equation	Name	D	Feasible Bounds
$f_1 = \sum_{i=1}^{D} x_i^2$	Sphere/Parabola	30	$(-100, 100)^D$
$f_2 = \sum_{i=1}^{D}(\sum_{j=1}^{i} x_j)^2$	Schwefel 1.2	30	$(-100, 100)^D$
$f_3 = \sum_{i=1}^{D-1} \left\{100 \left(x_{i+1} - x_i^2\right)^2 + (x_i - 1)^2\right\}$	Generalized Rosenbrock	30	$(-30, 30)^D$
$f_4 = -\sum_{i=1}^{D} x_i \sin\left(\sqrt{x_i}\right)$	Generalized Schwefel 2.6	30	$(-500, 500)^D$
$f_5 = \sum_{i=1}^{D} \left\{x_i^2 - 10\cos(2\pi x_i) + 10\right\}$	Generalized Rastrigin	30	$(-5.12, 5.12)^D$
$f_6 = -20 \exp\left\{-0.2\sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2}\right\} - \exp\left\{\frac{1}{D}\sum_{i=1}^{D}\cos(2\pi x_i)\right\} + 20 + e$	Ackley	30	$(-32, 32)^D$
$f_7 = \frac{1}{4000}\sum_{i=1}^{D} x_i^2 - \prod_{i=1}^{D}\cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	Generalized Griewank	30	$(-600, 600)^D$
$f_8 = \frac{\pi}{D}\left\{10\sin^2(\pi y_i) + \sum_{i=1}^{D-1}(y_i - 1)^2\left\{1 + 10\sin^2(\pi y_{i+1})\right\} + (y_D - 1)^2\right\}$	Penalized Function P8	30	$(-50, 50)^D$
$+\sum_{i=1}^{D}\mu(x_i, 10, 100, 4)$			
$y_i = 1 + \frac{1}{4}(x_i + 1)$			
$\mu(x_i, a, k, m) = \begin{cases} k(x_i - a)^m & x_i > a \\ 0 & -a \le x_i \le a \\ k(-x_i - a)^m & x_i < -a \end{cases}$			
$f_9 = 0.1\left\{\sin^2(3\pi x_i) + \sum_{i=1}^{D-1}(x_i - 1)^2\left\{1 + \sin^2(3\pi x_{i+1})\right\} + (x_D - 1)^2 \times\right.$	Penalized Function P16	30	$(-50, 50)^D$
$\left.\left\{1 + \sin^2(2\pi x_D)\right\}\right\} + \sum_{i=1}^{D}\mu(x_i, 5, 100, 4)$			
$f_{10} = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1 x_2 - 4x_2^2 + 4x_2^4$	Six-hump Camel-back	2	$(-5, 5)^D$
$f_{11} = \left\{1 + (x_1 + x_2 + 1)^2\left(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2\right)\right\} \times$	Goldstein-Price	2	$(-2, 2)^D$
$\left\{30 + (2x_1 - 3x_2)^2\left(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2\right)\right\}$			
$f_{12} = -\sum_{i=1}^{5}\left\{\sum_{j=1}^{4}(x_j - a_{ij})^2 + c_i\right\}^{-1}$	Shekel 5	4	$(0, 10)^D$
$f_{13} = -\sum_{i=1}^{7}\left\{\sum_{j=1}^{4}(x_j - a_{ij})^2 + c_i\right\}^{-1}$	Shekel 7	4	$(0, 10)^D$

Bratton, Kennedy, 2007: the paper has been provided to you

Best-of-M-runs: Parallelization

- If your Matlab installation has the Parallel computing toolbox (PCT), you can experiment with parallel computing
  - To check if you have PCT, run the command `ver` in Matlab
- The easiest parallelization is for tasks that run completely independently of each other: this is the case for best-of-M-runs
- For such tasks, a `for` loop can be replaced by a `parfor` loop; no other change is needed!
  - Read the documentation for `parfor` in Matlab
- The advantage of parallelization will only be apparent if your computer's CPU has multiple processing cores: you can find this out by looking at your system information (e.g., use "About my Mac" under the Apple icon on a Mac; Google the required steps for a Windows machine)
  - If you are using `parfor` for the first time, Matlab may ask you for some configuration information: set the default number of parallel workers to be the number of processing cores
- You can see `parfor` in action in **test_crcbpso_par.m** which does the same optimization as **test_crcbpso.m** but using 4 independent runs.

CRCBPSO: Additional features

```
%S=CRCBPSO(Fhandle,N,P)
%overrides the default PSO parameters with those provided in structure P.
%Set any of the fields of P to [] in order to invoke the corresponding
%default value. The fields of P are as follows.
%     'popSize': Number of PSO particles
%     'maxSteps': Number of iterations for termination
                                            ⋮
%     'boundaryCond': Set to '' for the "let them fly" boundary condition.
%                     Any other value is passed onto the fitness function
%                     for further processing. (NOT YET IMPLEMENTED PROPERLY)
%     'nbrhdSz' : Number of particles in a ring topology neighborhood.
%                 Reset to 3 if less than 3.
%Setting P to [] will invoke the default values for all pso parameters.
```

CRCBPSO: Additional features

```
%S = CRCBPSO(Fhandle,N,P,O)
%O is an integer that controls the amount of information returned in S. The
%default value of O is zero and returns S as the struct defined above.
%Progressively higher values of O increase the number of fields in S as
%listed below.
%   'allBestFit': O = 1. Best fitness values for all iterations returned as a
vector.
%   'allBestLoc': O = 2. Best locations in standardized coordinates for all
%                 iterations returned as a row of a matrix.
```

- GLRT with PSO

Plan

- Using PSO to get GLRT for the Quadratic Chirp in Colored Gaussian Noise

1. Develop code for the fitness function (**glrtqcsig.m**)
2. Develop code for calling PSO on the fitness function with multiple independent runs and picking the results from the best run
3. Create a data realization and use codes to obtain GLRT using PSO
4. Display results

Modify code of glrtqcsig.m following **crcbqcfitfunc** → Modify code of function calling PSO: **crcbqcpso (new name)** → Supply data to crcbqcpso and… magic!

Best-of-M runs

- Learn about Matlab's **parfor** command

crcbqcpso.m

```
53      %Independent runs of PSO in parallel. Change 'parfor' to 'for' if the
54      %parallel computing toolbox is not available.
55 -    parfor lpruns = 1:nRuns
56          %Reset random number generator for each worker
57 -        rng(lpruns);
58 -        outStruct(lpruns)=crcbpso(fHandle,nDim,psoParams);
59 -    end
```

- Note how the pseudo-random number generator is reset on line 57: Each worker will get a separate seed but the seed will be the same in every call to crcbqcpso
- Controlling PRNG seeds ensures reproducibility when using stochastic optimization methods

Fitness function

Preliminaries

- You have already coded the fitness function required for quadratic chirp in colored Gaussian noise
- You have to now **cast it into a form that can be called by the PSO code**
- This is best done by taking the fitness function already coded for WGN and modifying it by replacing some parts

Fitness function

F = ~~CRCBQCFITFUNC~~<glrtqcsig4pso>(X,P)

Compute the fitness function (~~sum of squared residuals function after maximimzation over the amplitude parameter~~→ log-likelihood ratio for colored noise maximized over the amplitude parameter) for data containing the quadratic chirp signal at the parameter values in X.

The fitness values are returned in F.

X is standardized, that is 0<=X(i,j)<=1.

The fields P.rmin and P.rmax are used to convert X(i,j) internally before computing the fitness: X(:,j) -> X(:,j)*(rmax(j)-rmin(j)) + rmin(j). $\Rightarrow \theta_i = x_i * (b_i - a_i) + a_i$

Fitness function

F = ~~CRCBQCFITFUNC~~<glrtqcsig4pso>(X,P)

The fields P.dataY and P.dataX are used to transport the data and its time stamps. The fields P.dataXSq and P.dataXCb contain the timestamps squared and cubed respectively. You will need an extra field to supply the PSD for colored noise.

[F,R] = ~~CRCBQCFITFUNC~~<glrtqcsig4pso> (X,P) returns the quadratic chirp coefficients corresponding to the rows of X in R. Converts standardized to real (unstandardized) coordinates, i.e., the parameters a1, a2, and a3

[F,R,S] = ~~CRCBQCFITFUNC~~<glrtqcsig4pso> (X,P) Returns the quadratic chirp signals corresponding to the rows of X in S. Converts the real coordinates to QC signal time series

```
% Only the body of this block should be replaced for different fitness functions
fitVal(lpc) = ssrqc(x, params);


function ssrVal = ssrqc(x,params)
```
← could call the fitness function glrtqcsig BUT…

```
%Generate normalized quadratic chirp
```

More efficient if the signal is generated inside this function for speed (calling crcbgenqcsig.m would be slow because dataX.^2 and dataX.^3 will be recomputed in every call but they need to be computed only once)

```
phaseVec = x(1)*params.dataX + x(2)*params.dataXSq + x(3)*params.dataXCb;
qc = sin(2*pi*phaseVec);
qc = qc/norm(qc);
```
⇒ norm is the one defined for WGN → Replace by the norm for given PSD; you have it in glrtqcsig.m; see DETEST/normsig4psd

```
%Compute fitness
ssrVal = -(params.dataY*qc')^2;
```
$\Rightarrow -\langle \bar{y}, \bar{q}(\Theta) \rangle^2$ for WGN → Replace by the inner product for given PSD; you have it in glrtqcsig.m; see DETEST/innerprodpsd

Test the **ssrqc** function independently… you can move it to a separate file and write a test script. For the same inputs it should give the same output as **glrtqcsig.m**.

Tasks

- Write a test script, test_glrtqcsig4pso.m, to do the following steps on glrtqcsig4pso.m.
  - You already have all the necessary codes from earlier exercises where you wrote glrtqcsig.m and then used it on data realizations data<n>.txt (n = 1, 2, 3) or on noise-only data realizations generated for a given PSD

1. Create a data realization with colored Gaussian noise and SNR=10 quadratic chirp signal.
   - Let the values of the parameters of this signal be $a_1^{(0)}$, $a_2^{(0)}, a_3^{(0)}$, the superscript denoting that these are true parameters
2. Create an **array of values**, A, for the parameter a_1 with minimum and maximum values a_1^{min} and a_1^{max}, respectively, and step size Δa_1
   - Ensure that $a_1^{(0)} \in [a_1^{min}, a_1^{max}]$
3. Similarly, set ranges $[a_2^{min}, a_2^{max}]$ and $[a_3^{min}, a_3^{max}]$, such that $a_2^{(0)} \in [a_2^{min}, a_2^{max}]$, $a_3^{(0)} \in [a_3^{min}, a_3^{max}]$ (But **no need to create an array** of values here)
4. Using the ranges above, create a matrix, X, such that its row i contains $[x_1^i, x_2^i, x_3^i]$, where
$$x_1^i = \frac{A(i) - a_1^{min}}{a_1^{max} - a_1^{min}}; \quad x_2^i = \frac{a_2^{(0)} - a_2^{min}}{a_2^{max} - a_2^{min}}; x_3^i = \frac{a_3^{(0)} - a_3^{min}}{a_3^{max} - a_3^{min}}$$
   - In other words, we have constructed standardized coordinate values where $x_j^i \in [0,1]$
5. Using the data realization created in step 1, compute the fitness values for X using glrtqcsig4pso.m
6. Plot the fitness values against the values in A: you should see the global minimum value near $a_1^{(0)}$

Calling PSO

CRCBQCPSO: Inputs

```
%O = CRCBQCPSO <New Name>(I,P,N)
%I is the input struct with the fields given below.
% 'dataY': The data vector (a uniformly sampled time series).
% 'dataX': The time stamps of the data samples.
% 'dataXSq': dataX.^2
% 'dataXCb': dataX.^3
```

New fields needed for colored Gaussian noise:

```
'psd' : PSD values at positive DFT frequencies
'sampFreq': Sampling frequency
% 'rmin', 'rmax': The minimum and maximum values of the three parameters
%                 a1, a2, a3 in the candidate signal:
%                 a1*dataX+a2*dataXSq+a3*dataXCb
```

```
P is the PSO parameter
%struct. Setting P to [] will invoke default parameters (see
CRCBPSO).

% N is the number of independent PSO runs.

%The output is returned in the struct O.
```

CRCBQCPSO: Outputs

```
%The fields of O are:
% 'allRunsOutput': An N element struct array containing results
                  from each PSO run. The fields of this struct are:
%     'fitVal': The fitness value.
%     'qcCoefs': The coefficients [a1, a2, a3].
%     'estSig': The estimated signal.
%     'totalFuncEvals': The total number of fitness evaluations.
% 'bestRun': The best run.
% 'bestFitness': best fitness from the best run.
% 'bestSig' : The signal estimated in the best run.
% 'bestQcCoefs' : [a1, a2, a3] found in the best run.
```

Signal from estimated parameters

```
%Prepare output
fitVal = zeros(1,nRuns);
for lpruns = 1:nRuns
    fitVal(lpruns) = outStruct(lpruns).bestFitness;
    outResults.allRunsOutput(lpruns).fitVal = fitVal(lpruns);
    [~,qcCoefs] = fHandle(outStruct(lpruns).bestLocation);
    outResults.allRunsOutput(lpruns).qcCoefs = qcCoefs;
    estSig = crcbgenqcsig(inParams.dataX,1,qcCoefs);
    estAmp = inParams.dataY*estSig(:);
    estSig = estAmp*estSig;
    outResults.allRunsOutput(lpruns).estSig = estSig;
    outResults.allRunsOutput(lpruns).totalFuncEvals = outStruct(lpruns).totalFuncEvals;
end
```

For each PSO run

Convert standardized coordinates to QC parameters a1, a2, a3

Signal from estimated parameters

```
%Prepare output
fitVal = zeros(1,nRuns);
for lpruns = 1:nRuns
    fitVal(lpruns) = outStruct(lpruns).bestFitness;
    outResults.allRunsOutput(lpruns).fitVal = fitVal(lpruns);
    [~,qcCoefs] = fHandle(outStruct(lpruns).bestLocation);
    outResults.allRunsOutput(lpruns).qcCoefs = qcCoefs;
    estSig = crcbgenqcsig(inParams.dataX,1,qcCoefs);
    estAmp = inParams.dataY*estSig(:);
    estSig = estAmp*estSig;
    outResults.allRunsOutput(lpruns).estSig = estSig;
    outResults.allRunsOutput(lpruns).totalFuncEvals = outStruct(lpruns).totalFuncEvals;
end
```

Store the QC parameters

Signal from estimated parameters

```
%Prepare output
fitVal = zeros(1,nRuns);
for lpruns = 1:nRuns
    fitVal(lpruns) = outStruct(lpruns).bestFitness;
    outResults.allRunsOutput(lpruns).fitVal = fitVal(lpruns);
    [~,qcCoefs] = fHandle(outStruct(lpruns).bestLocation);
    outResults.allRunsOutput(lpruns).qcCoefs = qcCoefs;
    estSig = crcbgenqcsig(inParams.dataX,1,qcCoefs);
    estAmp = inParams.dataY*estSig(:);
    estSig = estAmp*estSig;
    outResults.allRunsOutput(lpruns).estSig = estSig;
    outResults.allRunsOutput(lpruns).totalFuncEvals = outStruct(lpruns).totalFuncEvals;
end
```

Use crcbgenqcsig to obtain the QC signal (signal generation within glrtqcsig4pso must be identical to crcbgenqcsig except for normalization)

Signal from estimated parameters

```
%Prepare output
fitVal = zeros(1,nRuns);
for lpruns = 1:nRuns
    fitVal(lpruns) = outStruct(lpruns).bestFitness;
    outResults.allRunsOutput(lpruns).fitVal = fitVal(lpruns);
    [~,qcCoefs] = fHandle(outStruct(lpruns).bestLocation);
    outResults.allRunsOutput(lpruns).qcCoefs = qcCoefs;
    estSig = crcbgenqcsig(inParams.dataX,1,qcCoefs);
    estAmp = inParams.dataY*estSig(:);
    estSig = estAmp*estSig;
    outResults.allRunsOutput(lpruns).estSig = estSig;
    outResults.allRunsOutput(lpruns).totalFuncEvals = outStruct(lpruns).totalFuncEvals;
end
```

Caution: Normalization in crcbgenqcsig is for the case of WGN! Won’t work for colored noise

Signal from estimated parameters

```
%Prepare output
fitVal = zeros(1,nRuns);
for lpruns = 1:nRuns
    fitVal(lpruns) = outStruct(lpruns).bestFitness;
    outResults.allRunsOutput(lpruns).fitVal = fitVal(lpruns);
    [~,qcCoefs] = fHandle(outStruct(lpruns).bestLocation);
    outResults.allRunsOutput(lpruns).qcCoefs = qcCoefs;
    estSig = crcbgenqcsig(inParams.dataX,1,qcCoefs);
    estAmp = inParams.dataY*estSig(:);
    estSig = estAmp*estSig;
    outResults.allRunsOutput(lpruns).estSig = estSig;
    outResults.allRunsOutput(lpruns).totalFuncEvals = outStruct(lpruns).totalFuncEvals;
end
```

Replace!

1. Obtain estSig as shown,…
2. Normalize estSig again to have norm 1 according to the inner product $\langle \bar{x}, \bar{z} \rangle$ for <u>colored noise</u>
3. Obtain estimated amplitude: $A = \langle \bar{y}, \bar{q}(\Theta) \rangle$ (See derivation of GLRT in lecture)

Data realization

Data realization

- Number of samples: 512
- Sampling frequency: 512 Hz
- **Noise realization**: Toy PSD (**Note change of numbers**)

```
noisePSD = @(f) (f>=50 & f<=100).*(f-50).*(100-f)/625 + 1;
```

  - You will need to generate the PSD at positive DFT frequencies
- **Signal**: Quadratic chirp with the following parameters
  - SNR=10
  - $a_1 = 10, a_2 = 3, a_3 = 3$
- Data: **Noise realization** + **signal**

GLRT calculation

PSO search range

- Number of independent PSO runs = 8
- Number of iterations = 1000
- Search ranges
  - $a_1 \in [1, 180]$
  - $a_2 \in [1, 10]$
  - $a_3 \in [1,10]$
- $\Rightarrow$ We are searching over signals whose
  - **final instantaneous frequency** is in the range $[1 + 2 + 3, 180 + 20 + 30] = [6, 230]$ Hz
  - Initial instantaneous frequency is 1 Hz.
- This range is 89.4% of the widest possible range $[0, 256]$ Hz

Tasks

- Write a Matlab script to do the following:
  1. Generates the data realization as described in the “Data realization” slide
  2. Runs your modified version of `crcbqcpso` using the settings given in the “PSO search range” slide
  3. Makes a figure as follows:
     - Plots the data realization
     - Plots the true signal on top
     - Plots the best estimated signal on top
- (Optional) plot the best signals from each PSO run in a different color: This will show the spread in performance of PSO: see SDMBIGDAT19/test_crcbqcpso.m for inspiration!
- If you have done everything correctly, your best estimated signal should be a close match to the true signal